



UNIVERSIDAD ALFONSO X EL SABIO
BUSINESS&TECH FACULTY

STUDY OF THE MULTI-ARMED BANDIT PROBLEM AND ITS APPLICATIONS

Author: Eduardo López Pérez

Tutor: Javier Jiménez Raboso

MASTER'S THESIS

Master of Big Data analysis for business

Abstract

This document addresses the multi-armed bandit problem, focusing on the main algorithms traditionally used to solve it and their possible applications in real-world environments. The algorithms discussed are ϵ -Greedy, Upper Confidence Bound, Thompson Sampling, Linear Upper Confidence Bound, and Linear Thompson Sampling, and they are applied to the practical case of an e-commerce platform, which have become increasingly relevant with the digitization of our society. The accuracy of the recommendations made by the aforementioned algorithms is analysed based on whether or not users click on the products suggested by the algorithm. The increase in the accuracy of the recommendations when the user's context is considered to make suggestions is highlighted, and the possibility of using deep learning-based models to further improve the system's recommendations is discussed, which is left for future work.

Keywords: multi-armed bandit, ϵ -Greedy, Upper Confidence Bound, Thompson Sampling, Linear Upper Confidence Bound, Linear Thompson Sampling, reward, regret, e-commerce.

Contents

1	Introduction	1
2	Motivation and objectives	3
3	State of the art	4
4	Methodology	9
5	Results and discussion	13
6	Conclusions	21
A	Python code implementation	23

Chapter 1

Introduction

In this work it is intended to address the multi-armed bandit problem and its extension to contextual multi-armed bandits. This problem is one of the most common examples within reinforcement learning and has numerous applications in the real world. Of these applications, the most widespread are: the design of recommender systems (for streaming platforms, e-commerce, or social networks), clinical trials, dynamic routing, and the design of A/B tests.

In the case of recommender systems, the boom in the number of services that nowadays are carried out by most people through the internet has made these systems take on enormous importance when it comes to maximizing profits for multiple companies and, therefore, the application of algorithms for the resolution of the multi-armed bandit problem and reinforcement learning in general has gained a greater degree of relevance recently.

The majority of these services offered through the internet generate an enormous quantity of data about how users interact with a web page, which causes this type of application of the multi-armed bandit problem to be carried out in Big Data environments, bringing certain challenges, although also certain advantages, since reinforcement learning algorithms never stop learning, and as more data are generated (as long as these data are of good quality) these systems will continue improving more and more every day.

With respect to this work, it is intended to carry out a theoretical introduction to the multi-armed bandit problem and its extension to the contextual multi-armed bandit, and also to the main algorithms used to solve these problems, additionally showing how these algorithms would be used for the recommender system of an e-commerce platform.

The document will be structured in the following way:

- In a first section, the motivation and the objectives of the work will be presented.
- Next, a review of the literature will be made, in which the entire theoretical context of these problems and the main algorithms used to solve them will be explained.
- Afterward, there will be a section devoted to the methodology followed for the implementation of the practical part of the work, in which the algorithms presented in the state of the art will be applied to the recommender system on an e-commerce website.
- After the methodology, there will be a section of results and discussion, in which the results obtained from the application of these algorithms to the recommender system will be explained.

- And to conclude, there will be a conclusions section in which the general conclusions of the work will be presented.

Chapter 2

Motivation and objectives

The reason for the choice of the multi-armed bandit problem as the topic of this master's thesis is to carry out a study of a field of machine learning (reinforcement learning) that is normally not dealt with in much detail when one begins to study machine learning and artificial intelligence, since unsupervised learning, and above all supervised learning, usually take up more of the focus due to their greater number of applications in different areas.

The main objective of this work consists in showing the usefulness of the theoretical framework of multi-armed bandits for the resolution of real-world problems, such as the design of a recommender system for an e-commerce platform similar to what was done in Rohde et al. (2018), which will be showed in the practical part of this document.

The multi-armed bandit and contextual multi-armed bandit problems are problems that are usually studied from an academic point of view as a first example of reinforcement learning, but their transfer to real-world environments is almost never studied. These problems, despite their apparent simplicity, give rise to a broad and complex field of study, and whose application in business environments can have a high added value.

Chapter 3

State of the art

In this section an explanation will be given of what both the multi-armed bandit problem and the contextual multi-armed bandit problem consist of, and the main algorithms that have been used classically in the literature to address these problems will be explained in detail.

The multi-armed bandit problem arises when one is presented with n different options (or actions) repeatedly and intends to maximize the reward obtained from the chosen options over a stipulated period of time.

These rewards usually consist of numerical values coming from a stationary probability distribution that differs according to the action chosen.

The example most commonly used to explain this problem is that of a set of slot machines. Each slot machine translates into a different arm of the multi-armed bandit and the objective of the problem will be to decide which machine to choose to maximize the amount of money obtained from the machines' prizes. To maximize these prizes, one will try to choose those slot machines that have a higher probability of giving a prize.

Each of the actions of a multi-armed bandit has an expected average reward, which, if they were known for each one of the arms, would turn this problem into a trivial problem, since one would always choose the action with the best average reward. However, these expected average rewards are not known in most cases, or at least not exactly. For this reason, in order to estimate what the average reward of each of the actions is, it will be necessary to try choosing each one of them and to evaluate the rewards obtained so that an estimate can be made of the average of each possible option.

Obtaining a good balance between trying the different possible actions to improve the estimates of the average rewards of each of these actions and choosing the action with the best average reward according to the available estimates to maximize the reward obtained is something fundamental in this type of problems, and is what is known as the exploration versus exploitation dilemma or conflict.

This phenomenon of exploration vs. exploitation is a concept widely studied in the field of reinforcement learning, and there is no unique solution that allows solving this problem for any multi-armed bandit.

For this reason, there exists a large number of algorithms that are used nowadays to solve this class of problems, each of them balancing in a different way this equilibrium between

exploration and exploitation.

Generally speaking, algorithms with a greater degree of exploitation will obtain better accumulated rewards in short periods of time, while algorithms with a greater degree of exploration will obtain better accumulated rewards in longer periods of time, since the latter are more likely to find actions with better average rewards than the actions that at first were believed to have the best average rewards among all possible actions.

The most commonly used classical algorithms for solving multi-armed bandit problems are the ϵ -greedy algorithm, Upper Confidence Bound (UCB), and Thompson Sampling (TS). Below, the mathematics behind each of these algorithms will be explained. For more detailed information, Sutton and Barto (2018) can be consulted for the first two algorithms, and Russo et al. (2017) for Thompson Sampling.

ϵ -greedy algorithm: This algorithm or group of algorithms consists of choosing the action that is expected to give the best reward with probability $(1 - \epsilon)$. From a formal point of view one has that:

Given an action a , the estimate of the value of said action at a time step t will be

$$Q_t(a) = \frac{r_1 + r_2 + \dots + r_{n_t(a)}}{n_t(a)}$$

Where $r_1, r_2, \dots, r_{n_t(a)}$ are the rewards of the $n_t(a)$ previous time steps in which action a was chosen. This method to estimate the value of the actions is known as the sample-average method.

When $n_t(a) = 0$, a default value is assigned to $Q_t(a)$, and when $n_t(a) \rightarrow \infty$, by the law of large numbers one has that $Q_t(a) = q(a)$, where $q(a)$ is the true value of action a .

A *greedy* algorithm will choose at time t the action:

$$a_t = \operatorname{argmax}_a Q_t(a)$$

Where a_t is the action chosen at time t . In case there are several actions that maximize Q_t , the chosen action will be a random one among all that maximize it.

The ϵ -greedy algorithm will choose that action with probability $(1 - \epsilon)$, with ϵ being a fixed value belonging to the interval $[0, 1]$, and a different action chosen completely at random with probability ϵ .

When $\epsilon = 0$, we will be in the case of a completely *greedy* algorithm, and when $\epsilon = 1$, we will be in the case of an algorithm that chooses actions completely at random.

Upper Confidence Bound (UCB) algorithm: Contrary to the case of ϵ -greedy methods, the exploration of the actions that a priori are not optimal is carried out following a strategy different from choosing a random action among all possible ones. In the case of the UCB algorithm, the choice of the action to explore is made depending on the potential that each of the actions has to end up maximizing the reward, taking into account both how close the estimate of the mean of each action is to the action whose average reward is the maximum, and the uncertainty of such estimates.

The decision of which action to choose is made in this case following the formula:

$$a_t = \operatorname{argmax}_a \left[Q_t(a) + \alpha \sqrt{\frac{\ln t}{n_t(a)}} \right]$$

Where $\ln t$ denotes the natural logarithm of t and $\alpha > 0$ is an adjustable parameter that controls the degree of exploration of the algorithm. If $n_t(a) = 0$, a is considered to be maximizing the action, thus avoiding divisions by 0.

The term that is found inside the square root serves as a measure of the uncertainty of the estimation of the reward value of action a .

Thompson Sampling (TS) algorithm: This algorithm has a more probabilistic component than the previous algorithms, which are based on estimating the action to choose empirically and directly from the results of the actions chosen in previous time steps.

In this case, it is assumed that the rewards of the different actions to choose from follow probability distributions defined a priori and that are updated each time an action is chosen and a reward is obtained.

More formally: given an a priori probability distribution $p_0(a)$, for each action a , at each step of the Thompson Sampling algorithm a $\hat{Q}_t(a)$ is randomly sampled, where $p_t(a)$ is the a posteriori probability distribution at step t of action a . For that time t , the chosen action will be:

$$a_t = \operatorname{argmax}_a \hat{Q}_t(a)$$

The a posteriori probability distribution is obtained from $p_0(a)$, applying Bayes' theorem in the following way:

$$p(a|r_1, \dots, r_{n_t(a)}) = \frac{p(r_1, \dots, r_{n_t(a)}|a) \cdot p_0(a)}{p(r_1, \dots, r_{n_t(a)})}$$

Where a is the action for which the probability distribution is to be estimated, $r_1, \dots, r_{n_t(a)}$ are the rewards of the $n_t(a)$ previous time steps in which said action was chosen, and $p(a|r_1, \dots, r_{n_t(a)}) = p_t(a)$.

This update of the a posteriori distribution can be carried out at each time step by means of the recursive formula:

$$p_{t+1}(a) \propto p(r_t|a) \cdot p_t(a)$$

A widely known and studied extension of the multi-armed bandit problem is the so-called contextual multi-armed bandit. In this problem, in addition to the information available in a classical multi-armed bandit problem, there is also a context, which is known before making any decision and provides relevant information about the environment or the user, potentially helping to make a better decision regarding which action to choose.

This context, which will be denoted as $x_t \in X$, is usually given as a feature vector. The different available contexts are used to fit models (a different model for each available action) to predict the reward that will be obtained in each case.

Classically, the models used for this purpose were linear models, although nowadays more complex nonlinear models based on neural networks or decision trees are also used.

In this document, the Linear Upper Confidence Bound (LinUCB) and Linear Thompson Sampling (LinTS) algorithms will be covered, which are extensions of the previously presented algorithms to contextual multi-armed bandits through the use of linear algorithms to model the context. For more information about the first algorithm, see Zhou (2015), and for the second, Russo et al. (2017).

Linear Upper Confidence Bound (LinUCB) algorithm: It consists of an extension of the UCB algorithm through the use of linear models to represent the context. More rigorously, this algorithm can be stated as follows:

Given a context $x_{t,a} \in \mathbb{R}^d$ for each action a and each time t , the expected reward is assumed to be a linear function of the context in the following form:

$$Q_t(a) = x_{t,a}^\top \theta^*$$

Where $\theta^* \in \mathbb{R}^d$ is an unknown parameter vector which allows minimization of the loss function

$$R(T) = \sum_{t=1}^T \left(x_{t,a_t^*}^\top \theta^* - x_{t,a_t}^\top \theta^* \right)$$

Where $a_t^* = \operatorname{argmax}_{a \in A} x_{t,a}^\top \theta^*$.

This loss function is known as cumulative regret.

At each step of the algorithm, there is an estimate of θ^* obtained from the linear model. This estimate will be denoted as $\hat{\theta}_t$, and the action chosen at each step will be:

$$a_t = \operatorname{argmax}_a x_{t,a}^\top \hat{\theta}_{t-1} + \alpha \cdot \sqrt{x_{t,a}^\top A_{t-1}^{-1} x_{t,a}}$$

Where $\alpha > 0$ is the same adjustable parameter that controls the degree of exploration of the UCB algorithm, and the matrix A_{t-1} comes from the linear model, which consists of a ridge (regularized) regression of the following form:

$$\hat{\theta}_t = A_t^{-1} b_t$$

Where

$$A_t = \lambda I + \sum_{s=1}^t x_{s,a_s} x_{s,a_s}^\top \in \mathbb{R}^{d \times d}$$

$$b_t = \sum_{s=1}^t r_s x_{s,a_s} \in \mathbb{R}$$

$$A_0 = \lambda I$$

with λ being the trainable parameter of the linear regression, I the identity matrix, r_s the reward at time s , and x_{s,a_s} the context for action a_s at time s .

Linear Thompson Sampling (LinTS) algorithm: It is an extension of the Thompson Sampling algorithm through the use of linear models to account for context.

As in the previous algorithm, the expected reward is given by the equation:

$$Q_t(a) = x_{t,a}^\top \theta^*$$

However, in this case, $\hat{\theta}_t$, the estimate of the parameter vector θ^* , consists of a random sample from a posterior distribution over parameter vectors, $p_t(\theta)$, which is computed from a prior distribution $p_0(\theta)$, applying Bayes' theorem in a manner analogous to the Thompson Sampling algorithm, provided that this prior distribution is conjugate to the posterior, i.e., they belong to the same family. If this is not the case, numerical methods such as Markov Chain Monte Carlo (MCMC) must be used to compute the posterior distribution.

Therefore, taking $\hat{\theta}_t \sim p_t(\theta)$, the chosen action is:

$$a_t = \operatorname{argmax}_a x_{t,a}^\top \hat{\theta}_t$$

Finally, it is worth mentioning that extending these linear models to incorporate context into nonlinear models consists of replacing the linear expression $Q_t(a) = x_{t,a}^\top \theta^*$ with a nonlinear expression $Q_t(a) = f^*(x_{t,a})$, where f^* is a nonlinear function.

However, this transition from a linear function to a nonlinear one entails a series of theoretical complications that will not be addressed in this work.

Chapter 4

Methodology

As previously mentioned, the practical part of this work will consist of applying multi-armed bandits to an e-commerce platform. This section will detail the procedure followed to carry out this use case.

As is common in reinforcement learning applications, instead of directly using a dataset to train the implemented models, a simulator or bot was built against which the algorithms are used; that is, this bot generates, at each time step (or turn), a reward for the action chosen by the algorithm being used. Once the reward is returned, the policy of the multi-armed bandit algorithm is updated with respect to its preferences over which action to choose in each case, and the process moves to the next turn, where the simulator generates a new user, the algorithm suggests the most suitable action for that user, and the simulator returns the reward for said action.

In this case, the user corresponds to a user who has accessed the website, the actions are the possible products to recommend, and the reward is whether or not the user clicks on the recommended product in order to make a purchase.

The simulator used is self-developed, and its operation is as follows:

- First, a function is used to generate a set of users who will be the potential customers. These users will have an associated id, age, gender, and preferred product category.
 - The id is a number that will allow users to be uniquely identified.
 - The age will be generated as a random integer between 15 and 70.
 - the gender will be a random choice between male (M) and female (F).
 - And the preferred category will be randomly selected from among five categories: technology, fashion, books, sports, and home.
- A different function is used to generate the products available on the e-commerce platform. These products will have an associated id, a category, a base click rate received by the product on the platform, and an age factor.
 - In this case, the id will also be an integer number that will allow products to be uniquely identified.

- The category is the section to which the product belongs. As mentioned for the user generator, the possible categories will be: technology, fashion, books, sports, and home. The simulator is designed such that all categories have the same number of products.
- The base click rate defines the base probability (prior to having any type of information about the customer or the time of access to the website) that a given product is clicked if it is recommended to a customer. These probabilities are generated from a uniform distribution with lower bound 0.15 and upper bound 0.3, $U(0.15, 0.3)$.
- The age factor estimates how attractive a product is to customers according to their ages. This factor adds a term that follows a linear function to the base click rate (more information about this will be given later). The values of this factor come from random samples of a normal distribution $N(0.1, 0.05)$, bounded below by 0.02, meaning that if any of the sampled values is below 0.02, it is set to that value, and bounded above by 0.4.
- The third and final component of the simulator is the interaction simulator, which is responsible for deciding, for each user-product interaction, whether the user clicked on the recommended product or not.

This decision of whether the user clicked on the recommended product is made by sampling from a Bernoulli distribution, $Bernoulli(p)$, whose probability is computed as will be explained below:

For each product, as mentioned previously, there is a base click rate whose values come from samples of a continuous uniform distribution $U(0.15, 0.3)$.

This base click rate is multiplied by a factor depending on whether the customer's preferred product category matches the product category. This factor is 1.5 if they match and 0.75 if they do not.

It is also multiplied by a gender factor depending on the category of the suggested product. If the user is male and the suggested product belongs to the sports or technology categories, it is multiplied by a factor of 1.1; if it belongs to the other three categories, the factor is 0.9. If the user is female, the factors are reversed: 0.9 for sports and technology products, and 1.1 for the other three categories.

To this click rate, two additional terms are also added, one of which follows a linear function depending on the user's age and the other follows another linear function depending on the time at which the user accessed the website.

Therefore, the final click rate is given by the expression:

$$ctr = base \cdot cat \cdot gen + age + hour$$

where *base* is the base click rate, $cat \in \{0.75, 1.5\}$ is the category-matching factor described above, $gen \in \{0.9, 1.1\}$ is the gender factor also previously described, *age* is the linear term depending on the user's age and *hour* is the linear term depending on the time at which the user accessed the page.

The age term is given by the expression:

$$age = \frac{a \cdot x}{70}$$

where $a \in N(0.1, 0.05)$ is the coefficient of the linear regression and x is the user's age. The value of a is also constrained to lie within the interval $[0.02, 0.4]$.

The hour term is given by the expression:

$$hour = \frac{0.05 \cdot x}{24}$$

where in this case x is the hour of access to the website.

After designing this simulator, the next step was to use it as a data generator to train the reinforcement learning algorithms presented in the state of the art section.

For the case of the multi-armed bandit algorithms that do not use context (ϵ -Greedy, UCB, and TS), this was ignored, whereas for the contextual multi-armed bandit algorithms (LinUCB and LinTS), the context was indeed used. However, since this context contains categorical variables (user gender, user preferred product category, and product category), these variables had to be transformed, as they cannot be directly used to train the linear models underlying these algorithms.

These variables were encoded using One-Hot encoding, which consists of creating a binary variable for each category of one of these categorical variables, such that the variable takes the value 1 if the selected data point belongs to that category and 0 otherwise.

There are other alternative ways of encoding these categorical variables, such as ordinal encoding, in which each category of a variable is assigned a different number. However, One-Hot encoding was chosen because it does not introduce any bias between the different categories of a variable, unlike ordinal encoding, which imposes an ordering on the categories.

Before performing a comparison of the different models, the hyperparameters of the ϵ -Greedy, Upper Confidence Bound, Linear Upper Confidence Bound, and Linear Thompson Sampling algorithms were individually optimized in order to obtain the best possible version of each of these models for a fair comparison.

Note that the hyperparameters of the Thompson Sampling algorithm were not optimized. This is because this algorithm lacks numerical hyperparameters similar to those of the other algorithms. In this case, the only hyperparameter is the choice of prior distribution used, which was fixed to a beta distribution.

After this hyperparameter optimization, all models were trained simultaneously, such that the user selected at each time step of the problem was the same for all algorithms. For this user, whether they would click on each of the different products available on the platform was simulated, assuming each of them was recommended to them. Once this was done, the models were called to produce their recommendations, and the result of the suggested product was returned. Using the outcome of each algorithm's recommendation, they were updated (each one only with the result of its own suggested product), and the process moved on to the next iteration.

To evaluate the performance of all these algorithms, both cumulative reward and cumulative regret were used. In addition to these metrics, accuracy (number of times the best possible action was suggested divided by the number of time steps) and the proportion of visits that resulted in the user clicking on the suggested product (success rate) were also analyzed.

Chapter 5

Results and discussion

To measure the performance of all the algorithms introduced previously in the practical part, the simulator presented in the previous section was used.

For these tests, a dataset of 200 different users was generated. At each time step of the algorithms, one of these users was randomly selected as the one who accessed the website at that moment (this sampling was performed with replacement, meaning that the same user could be selected an unlimited number of times).

In addition, another dataset of 10 different products to be recommended was generated, with two products from each of the previously introduced categories.

Lastly, the simulator was used with these datasets to check whether the recommendations of the different algorithms succeeded in making the user click on the suggested product. This was carried out over 25,000 time steps with all algorithms running simultaneously, i.e., 25,000 website visits were simulated, and it was checked whether the recommendations generated by each of the algorithms resulted in the user clicking on the product or not. This process was repeated 5 times, and the metrics from these iterations were averaged and analyzed.

Before comparing the algorithms with each other, this same procedure was followed for hyperparameter optimization of the different algorithms. In the case of the ϵ -Greedy algorithm, values of the parameter ϵ from 0.01 to 1 were tested. Among all these values, $\epsilon = 0.1$ was the value that achieved the best cumulative reward. The results of these tests with different values of the ϵ parameter can be observed in Figure 5.1.

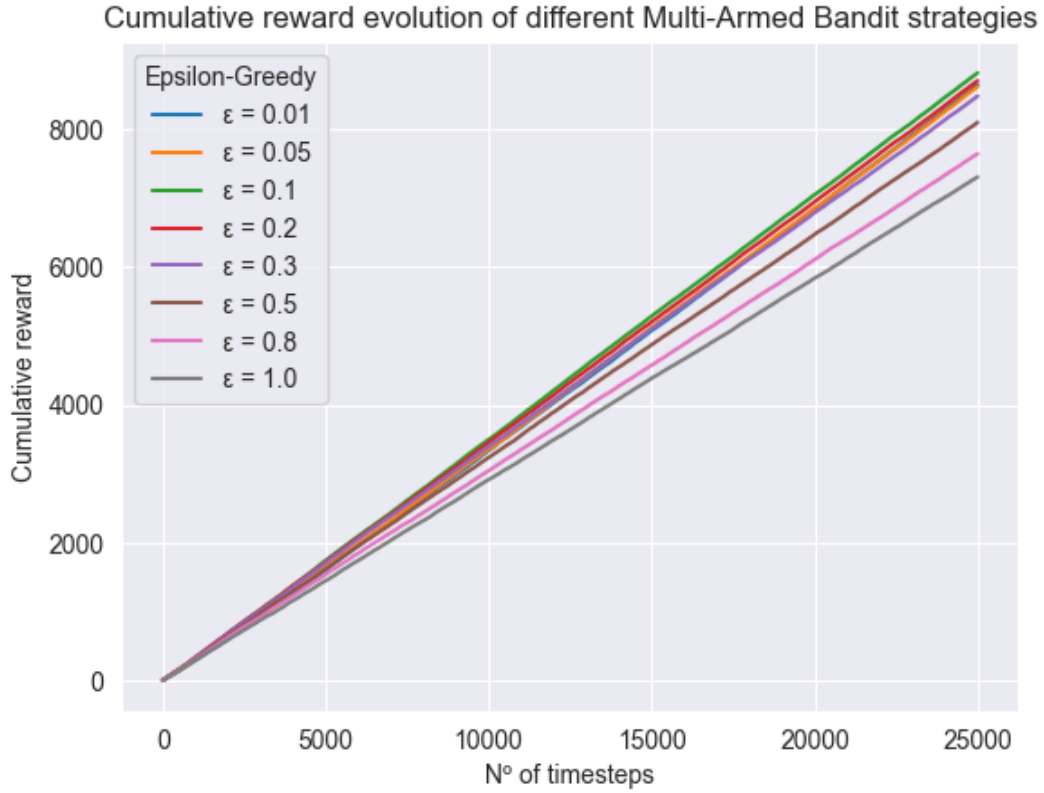


Figure 5.1: Graph showing the evolution of cumulative reward over time for different values of the parameter ϵ of the ϵ -Greedy algorithm.

For the UCB algorithm, values of the parameter α from 0.05 to 10 were tested, and the best results were obtained for $\alpha = 0.25$ and $\alpha = 0.5$. Between these two values, $\alpha = 0.25$ was ultimately chosen for use in the comparison between the different algorithms. The results of this hyperparameter optimization can be seen in Figure 5.2.

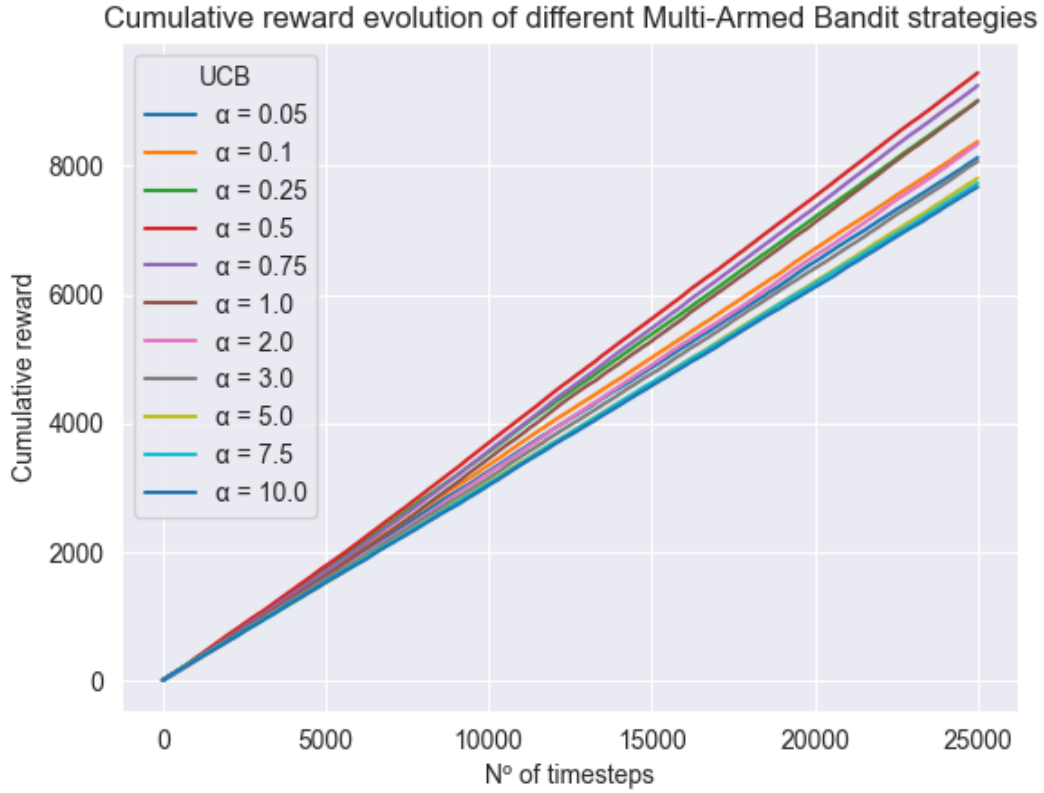


Figure 5.2: Graph showing the evolution of cumulative reward over time for different values of the parameter α of the UCB algorithm.

For the LinUCB algorithm, it was found that in this case the values of α that yielded the best results were $\alpha = 0.75$ and $\alpha = 1$, among all those tested (the same values as for UCB were tested). Between these two values, $\alpha = 0.75$ was chosen for the comparison between the different algorithms. The results of these tests can be seen in Figure 5.3.

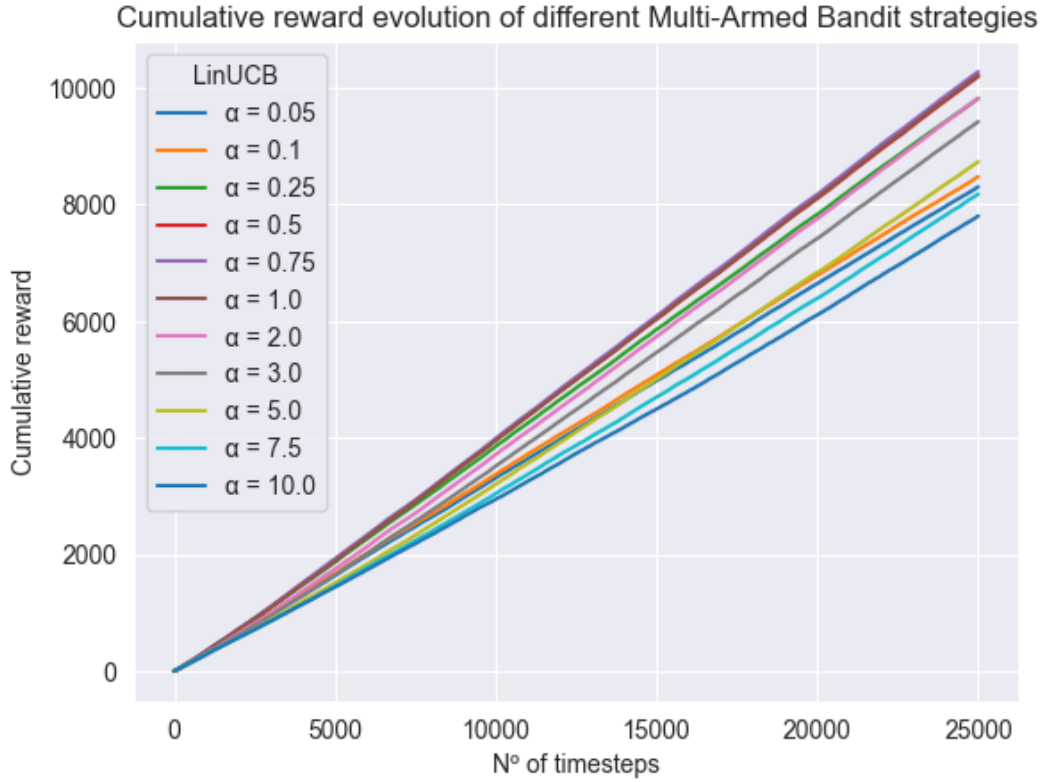


Figure 5.3: Graph showing the evolution of cumulative reward over time for different values of the parameter α of the LinUCB algorithm.

And for the LinTS algorithm, values of the parameter ν between 0.1 and 5 were tested, with the best results being obtained for $\nu = 0.3$. These results are shown in Figure 5.4.

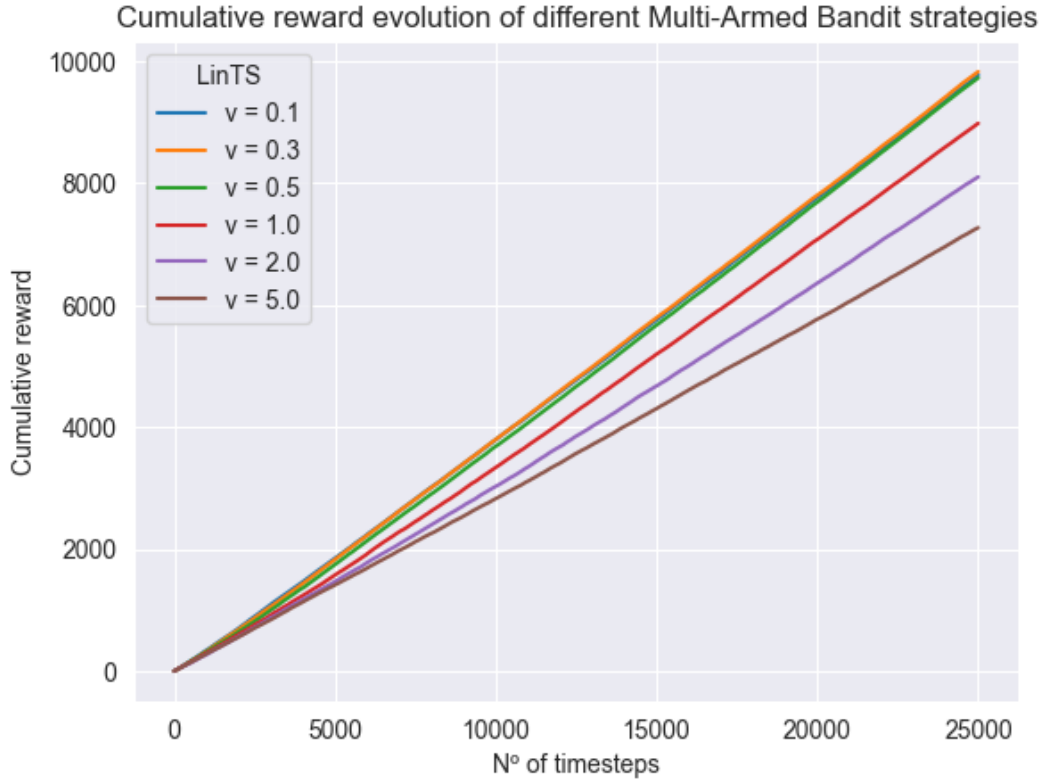


Figure 5.4: Graph showing the evolution of cumulative reward over time for different values of the parameter ν of the LinTS algorithm.

Once the hyperparameters of all algorithms had been optimized, a comparison between the algorithms was carried out. For this purpose, the previously mentioned parameter values were used, and the values of the metrics obtained from each of the algorithms were compared.

Regarding the evolution of cumulative rewards, in the graph shown in Figure 5.5 it can be observed that, as expected, contextual algorithms are superior to those that ignore context when such context is available, and all of these algorithms, with or without context, clearly outperform selecting products to recommend completely at random. In particular, the LinTS algorithm achieves slightly better results than LinUCB, making it the best among all those studied for this specific case.

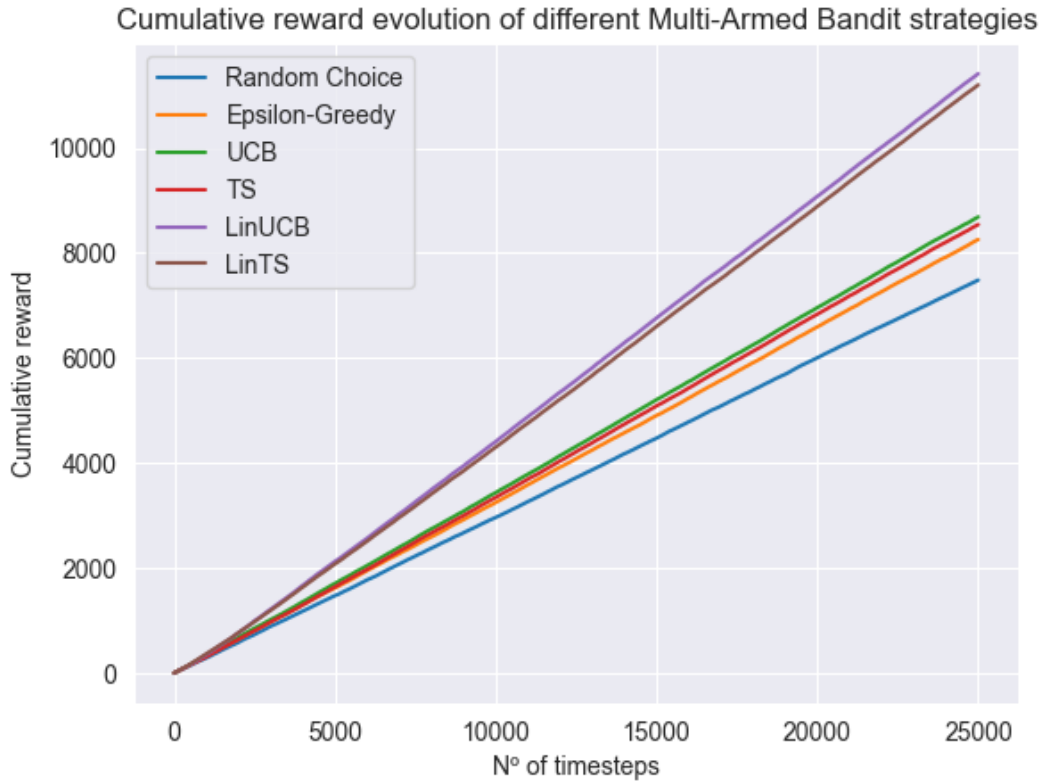


Figure 5.5: Graph showing the evolution of cumulative reward over time for the different algorithms used.

On the other hand, if the cumulative regret is observed (Figure 5.6), the ranking from best to worst performance of the algorithms was practically the same (note that in the case of cumulative regret, the goal is for it to be as low as possible). The algorithm with the lowest cumulative regret was also LinTS, closely followed by LinUCB, and with a clear advantage over the rest of the algorithms.

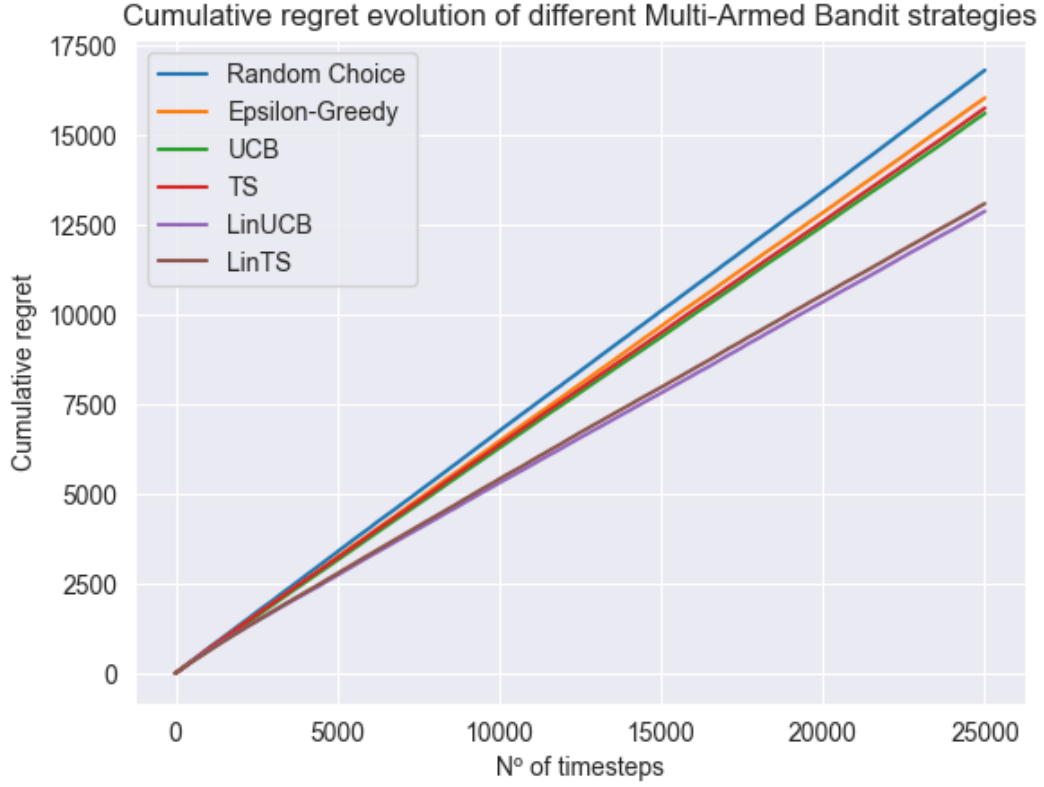


Figure 5.6: Graph showing the evolution of cumulative regret over time for the different algorithms used.

In addition to these two metrics, the success rate and the accuracy of the algorithms were also measured, and the results can be seen in Table 5.1. For these metrics, LinTS was also the algorithm that achieved the best results.

Algorithm	Success rate	Accuracy
Random choice	0,261	0,099
Epsilon-Greedy	0,336	0,283
UCB	0,349	0,321
TS	0,344	0,307
LinUCB	0,395	0,657
LinTS	0,401	0,744

Table 5.1: Results of the success rate and accuracy of all the algorithms used.

Regarding cumulative regret, it should be noted that in this case it was computed from empirical data using the following formula:

$$R(T) = \sum_{t=1}^T (\hat{r}_t - r_t)$$

where r_t is the reward of the algorithm selected at that time step and $\hat{r}_t$ is, among all possible rewards that could have occurred at that time step given the recommended product, the highest one.

The use of this alternative formula for computing cumulative regret is due to the fact that the theoretical formula introduced previously in the state of the art cannot be directly applied in practice in most cases, since in some situations the action with optimal reward is unknown, or, as in this case, due to the stochastic nature of the process (as it involves sampling from a Bernoulli distribution), the optimal action, defined as the one with the highest probability of a successful recommendation, may not be successful, while another action with lower probability of success may actually succeed. This could lead to regret, when computed using the theoretical formula, becoming negative at some time step, which is not possible by definition, as explained in Loomes and Sugden (1982).

Chapter 6

Conclusions

This document has presented a study of the multi-armed bandit problem and its extension to contextual multi-armed bandits, providing a detailed explanation of the main algorithms traditionally used to solve them.

These types of problems, as shown in this work, can be transferred from the theoretical domain to the practical domain in multiple areas, and are therefore highly useful.

In the case of using these algorithms for the implementation of recommendation systems, this represents a clear improvement over making recommendations in a completely random manner, which explains the rise of such systems in e-commerce platforms, social networks, and others.

Regarding the algorithms implemented in the designed recommendation system, they either ignore context entirely or model it as a linear function, which in many practical cases does not accurately reflect reality.

Nowadays, the state of the art in recommendation systems is increasingly shifting towards the use of more complex deep learning models. However, the use of these classical models will always have a place in certain applications, especially those where the available context is very limited or nonexistent, situations in which deep learning struggles to achieve good performance. For this reason, the study of these techniques remains of vital importance today.

As for deep learning techniques that can be used to solve the multi-armed bandit problem, their study and performance comparison with the techniques analyzed in this document is left for future work.

Bibliography

- Loomes, G., & Sugden, R. (1982). Regret theory: An alternative theory of rational choice under uncertainty. *The Economic Journal*, 92(368), 805–824. <https://doi.org/10.2307/2232669>
- Rohde, D., Bonner, S., Dunlop, T., Vasile, F., & Karatzoglou, A. (2018). Recogym: A reinforcement learning environment for the problem of product recommendation in online advertising. <https://arxiv.org/abs/1808.00720>
- Russo, D., Roy, B. V., Kazerouni, A., & Osband, I. (2017). A tutorial on thompson sampling. *CoRR*, abs/1707.02038. <http://arxiv.org/abs/1707.02038>
- Sutton, R., & Barto, A. (2018). *Reinforcement learning, second edition: An introduction*. MIT Press. <https://books.google.es/books?id=sWV0DwAAQBAJ>
- Zhou, L. (2015). A survey on contextual multi-armed bandits. *CoRR*, abs/1508.03326. <http://arxiv.org/abs/1508.03326>

Appendix A

Python code implementation

This appendix presents the Python code used for the implementation of the simulator, the ϵ -Greedy, UCB, TS, LinUCB, and LinTS algorithms, and the training of these models using the data generated by the simulator. All this code is available in the GitHub repository accessible at the following URL: <https://github.com/Eduard0LP/MABE-Multi-Armed-Bandits-for-E-commerce>.

The implementation of all code related to the models and the simulator was carried out using only the NumPy and Pandas libraries as dependencies. In addition to these libraries, scikit-learn was also used for encoding categorical variables into numerical variables during the training of the linear regressions of the contextual algorithms, and Matplotlib and Seaborn were used for the graphical representation of the results.

Regarding the simulator, the function that generates the users who will be the potential customers of the e-commerce platform was called *generate_user* and returns the data of a user in a Python dictionary. Its implementation can be seen in Figure A.1.

```
def generate_user(user_id: int, product_categories: list[str]) -> dict[str, Any]:
    """Function to generate a user for an e-commerce platform.

    This function can be used to generate a user for an e-commerce platform so that a Multi-Armed Bandit algorithm
    can be trained. It has to be combined with the functions generate_products and simulate_interactions in
    order to generate the final dataset that will be fed to the Multi-Armed Bandit algorithms.

    Args:
        user_id: ID that will be given to the user.
        product_categories: Possible options for the preferred_category field of the user dictionary.

    Returns:
        A dictionary containing the following fields:
        - user_id: The user_id passed as argument to the function.
        - age: Age of the user. It is an integer between 15 and 70 generated randomly.
        - gender: Gender of the user. Randomly chosen between male or female (M or F).
        - preferred_category: Preferred category of products of the user. It can be any given category from the
          product_categories list that is passed as argument to the function.
    """
    return {
        'user_id': user_id,
        'age': np.random.randint(15, 70),
        'gender': np.random.choice(['M', 'F']),
        'preferred_category': np.random.choice(product_categories)
    }
```

Figure A.1: Python function used to generate the website users.

The products were generated using the *generate_products* function (Figure A.2), which returns a Pandas DataFrame containing the data of all available products.

```
def generate_products(n_products: int, product_categories: list[str]) -> pd.DataFrame:
    """Funtion to generate the products for an e-commerce platform.

    This function can be used to generate the products for an e-commerce platform so that a Multi-Armed Bandit
    algorithm can be trained. It has to be combined with the functions generate_user and
    simulate_interactions in order to generate the final dataset that will be fed to the Multi-Armed Bandit algorithms.

    Args:
        n_products: Number of products that will be generated.
        product_categories: Possible options for the category column of the products dataframe.

    Returns:
        A pandas dataframe containing the following columns:
        - product_id: ID to identify the product.
        - category: Category the product belongs to.
        - base_ctr: Base rate/probability at which the product gets clicked when showed to a user. It is obtained
          from sampling a uniform distribution U(0.15, 0.3).
        - age_aggregate_factor: Coefficient of an additive factor dependant on age that is added to the base_ctr.
          age_aggregate is a linear function with respect to the age of the user that gets added to the base_ctr
          probability. It is obtained from a sample of a gaussian distribution N(0.1, 0.05) and clipping this
          value to a minimum of 0.02.
    """
    products = []
    for i in range(n_products):
        products.append({
            'product_id': i,
            'category': product_categories[int(i//(n_products/len(product_categories)))],
            'base_ctr': np.random.uniform(0.15, 0.3),
            'age_aggregate_factor': np.clip(np.random.normal(0.1, 0.05), 0.02, 0.4)
        })
    return pd.DataFrame(products)
```

Figure A.2: Python function used to generate the website products.

```
# Generar usuarios y productos
users = [generate_user(i) for i in range(n_users)]
products_df = generate_products()
```

Figure A.3: Code snippet in which the *generate_user* and *generate_products* functions are called to generate the users and the products.

The third and final component of the simulator, as mentioned in the methodology section, was responsible for simulating the interactions between users and products. This was implemented with the *simulate_interactions* function, which receives the user data generated by *generate_user* and the DataFrame generated by *generate_products*, and simulates whether that user would click or not on each product in the DataFrame if it were recommended to them. The results of all these simulations are returned in a new DataFrame. The implementation of this function can be seen in Figure A.4.

```

def simulate_interactions(user: dict[str, Any], products_df: pd.DataFrame, timestamp: pd.Timestamp,
                          cat_splits: tuple[list[str], list[str]] | None = None) -> pd.DataFrame:
    """Funtion to simulate the interactions between a user and all available products.

    This function simulates for a given user, if he/she would click in each one of the products of the
    products_df dataframe if each of these products was recommended to him/her by a Multi-Armed Bandit
    algorithm. It has to be combined with the functions generate_user and generate_products to be used as a complete
    simulator.

    The probability of clicking on a product if recommended is given by the following expression:

        base_ctr * category_multiplier * gender_multiplier + age_aggregate + hour_aggregate

    where:
    - base_ctr is the value given in the dataframe generated by generate_products.
    - category_multiplier is a multiplicative factor that has value 1.5 if the category of the product matches
      the preferred category of the user and 0.75 if not.
    - gender_multiplier is a multiplicative factor that takes value 1.1 if the category of the product is between
      the preferred categories of that gender, and 0.9 if not.
    - age_aggregate is an additive factor given by the linear expression age_aggregate_factor * age / 70. The
      age_aggregate_factor is the one given in the dataframe generated by generate_products.
    - hour_aggregate is an additive factor given by the linear expression 0.05 * timestamp.hour / 24.

    Args:
    user: User data. It is given by the output of the generate_user function.
    products_df: Dataframe with all the product characteristics. It is given by the output of the generate_products
    function.
    timestamp: Time at which the user accessed the platform.
    cat_splits: Optional parameter consisting of a tuple with 2 lists. The first list gives the preferred
    categories of the male users and the second list the preferred categories of the female users. If not provided
    the function _split_randomly is called to generate this tuple.

    Returns:
    A pandas dataframe containing the following columns:
    - product_id: ID of the product. It is copied from the column with the same name on products_df.
    - ctr: Probability of the binomial distribution used to decide whether the user clicked or not on a
      product if recommended.
    - click: Binary variable stating whether the user clicked or not on the product with that
      product_id if recommended to the user. It is obtained from a sample of a a binomial
      distribution B(1, ctr).
    """
    age_aggregate = products_df['age_aggregate_factor'].values * user['age'] / 70

    category_multiplier = np.where(products_df['category'].values == user['preferred_category'], 1.5, 0.75)

    if cat_splits is None:
        cat_splits = _split_randomly(products_df['category'].unique())

    if user['gender'] == 'M':
        condition = np.isin(products_df['category'].values, cat_splits[0])
    elif user['gender'] == 'F':
        condition = np.isin(products_df['category'].values, cat_splits[1])
    else:
        condition = np.zeros(len(products_df), dtype=bool)

    gender_multiplier = np.where(condition, 1.1, 0.9)

    hour_aggregate = 0.05 * timestamp.hour / 24

    ctr = products_df['base_ctr'].values * category_multiplier * gender_multiplier + age_aggregate + hour_aggregate

    click = np.random.binomial(1, ctr)

    return pd.DataFrame({'product_id': products_df['product_id'], 'ctr': ctr, 'click': click})

```

Figure A.4: Python function to simulate product–customer interactions.

Regarding the implementations of the algorithms discussed in this work, these were implemented using Python classes with three main methods: an `__init__` method to initialize the class with the appropriate parameterization of the algorithm, a `select_action` method through which the algorithm makes the recommendation of a product based on the context, and an `update` method to update the algorithm’s parameters after receiving the result of whether the recommendation received a reward or not. It is important to emphasize that even for algorithms that do not use context, it is still passed to the `select_action` and `update`

methods, and it is simply ignored within those classes.

This design choice was made to maintain exactly the same interface across all models, thus facilitating the simultaneous training of all of them within a single training loop using the simulator. The implementations of all these models can be seen in Figures A.5, A.6, A.7, A.8, and A.9. Regarding the training of all models, it was carried out jointly in the training loop shown in Figure A.10.

```
class EpsilonGreedy:
    """Class containing an implementation of the Epsilon-Greedy algorithm for Multi-Armed Bandits.

    This implementation initialises the average rewards of all the arms at 0, and updates them after each time step.

    Attributes:
        epsilon: Exploration parameter of the model. Defines the probability with which the model chooses an action
        different from the one that is expected to be the best.
        n_arms: Number of arms of the bandit problem.
        q_values: Vector with the experimental average rewards of all arms.
        action_counts: Vector with the count of how many times each action has been chosen.
    """

    def __init__(self, n_arms: int, epsilon: float = 0.1) -> None:
        """Initialisation of a class instance.

        Args:
            epsilon: Exploration parameter of the model. Defines the probability with which the model chooses an action
            different from the one that is expected to be the best.
            n_arms: Number of arms of the bandit problem.
        """
        self.epsilon = epsilon
        self.n_arms = n_arms
        self.q_values = np.zeros(n_arms)
        self.action_counts = np.zeros(n_arms)

    def select_action(self, x_contexts: np.ndarray) -> Union[int, np.int64]:
        """Method to choose an action or arm following the algorithm policy.

        This method suggests which arm to choose following the equations of the Epsilon-Greedy algorithm.

        Args:
            x_contexts: Vector with the context for an specific time step. It is ignored given that the algorithm
            is not contextual.

        Returns:
            The number of the arm that should be chosen according to the algorithm.
        """
        if np.random.rand() < self.epsilon:
            return np.random.choice(self.n_arms)
        return np.argmax(self.q_values)

    def update(self, action: int, x: np.ndarray, reward: float) -> None:
        """Update method of the model.

        This method is responsible for updating the parameters of the model after each time step (after an action is
        selected and the reward of that choice is returned).

        Args:
            action: Action or arm chosen.
            x: Context associated to that time step.
            reward: Reward returned from the action chosen.
        """
        self.action_counts[action] += 1
        n_t_a = 1 / self.action_counts[action]
        self.q_values[action] += n_t_a * (reward - self.q_values[action])
```

Figure A.5: Python class implementing the ϵ -Greedy algorithm.

```

class UCB:
    """Class containing an implementation of the Upper Confidence Bound (UCB) algorithm for Multi-Armed Bandits.

    This implementation initialises the average rewards of all the arms at 0, and updates them after each time step.

    Attributes:
        n_arms: Number of arms of the bandit problem.
        q_values: Vector with the experimental average rewards of all arms.
        action_counts: Vector with the count of how many times each action has been chosen.
        total_counts: Total amount of time steps the algorithm has taken.
        alpha: Parameter of the UCB algorithm that controls exploration.
    """
    def __init__(self, n_arms: int, alpha: float = 1.0) -> None:
        """Initialisation of a class instance.

        Args:
            n_arms: Number of arms of the bandit problem.
            alpha: Parameter of the UCB algorithm that controls exploration.
        """
        self.n_arms = n_arms
        self.q_values = np.zeros(n_arms)
        self.action_counts = np.zeros(n_arms)
        self.total_counts = 0
        self.alpha = alpha

    def select_action(self, x_contexts: np.ndarray) -> np.int64:
        """Method to choose an action or arm following the algorithm policy.

        This method suggests which arm to choose following the equations of the UCB algorithm.

        Args:
            x_contexts: Vector with the context for an specific time step. It is ignored given that the algorithm
                is not contextual.

        Returns:
            The number of the arm that should be chosen according to the algorithm.
        """
        self.total_counts += 1
        # The value 1e-5 is a small value used to avoid dividing by 0
        ucb_values = self.q_values + self.alpha * np.sqrt(np.log(self.total_counts) / (self.action_counts + 1e-5))
        return np.argmax(ucb_values)

    def update(self, action: int, x: np.ndarray, reward: float) -> None:
        """Update method of the model.

        This method is responsible for updating the parameters of the model after each time step (after an action is
        selected and the reward of that choice is returned).

        Args:
            action: Action or arm chosen.
            x: Context associated to that time step.
            reward: Reward returned from the action chosen.
        """
        self.action_counts[action] += 1
        n_t_a = 1 / self.action_counts[action]
        self.q_values[action] += n_t_a * (reward - self.q_values[action])

```

Figure A.6: Python class implementing the Upper Confidence Bound algorithm.

```

class ThompsonSampling:
    """Class containing an implementation of the Thompson Sampling (TS) algorithm for Multi-Armed Bandits.

    This implementation considers a beta distribution  $B(\alpha, \beta)$  as the prior distribution of the rewards.
    The parameters of the distribution are both initialised to 1.

    Attributes:
        n_arms: Number of arms of the bandit problem.
        successes: Vector with the amount of positive rewards of the suggestions of each arm plus one. This corresponds
        to the alpha parameter of the beta distribution.
        failures: Vector with the amount of negative rewards (no reward) of the suggestions of each arm plus one. This
        corresponds to the beta parameter of the beta distribution. .
    """
    def __init__(self, n_arms: int) -> None:
        """Initialisation of a class instance.

        Args:
            n_arms: Number of arms of the bandit problem.
        """
        self.n_arms = n_arms
        self.successes = np.ones(n_arms)
        self.failures = np.ones(n_arms)

    def select_action(self, x_contexts: np.ndarray) -> np.int64:
        """Method to choose an action or arm following the algorithm policy.

        This method suggests which arm to choose following the equations of the TS algorithm.

        Args:
            x_contexts: Vector with the context for an specific time step. It is ignored given that the algorithm
            is not contextual.

        Returns:
            The number of the arm that should be chosen according to the algorithm.
        """
        sampled_means = np.random.beta(self.successes, self.failures)
        return np.argmax(sampled_means)

    def update(self, action: int, x: np.ndarray, reward: float) -> None:
        """Update method of the model.

        This method is responsible for updating the parameters of the model after each time step (after an action is
        selected and the reward of that choice is returned).

        Args:
            action: Action or arm chosen.
            x: Context associated to that time step.
            reward: Reward returned from the action chosen.
        """
        if reward == 1:
            self.successes[action] += 1
        else:
            self.failures[action] += 1

```

Figure A.7: Python class implementing the Thompson Sampling algorithm.

```

class LinUCB:
    """Class containing an implementation of the Linear Upper Confidence Bound (LinUCB) algorithm for Contextual
    Multi-Armed Bandits.

    This implementation initialises the linear regression coefficient matrices to the identity matrix and the vectors of
    independent terms to 0, and updates them after each time step.

    Attributes:
        n_arms: Number of arms of the bandit problem.
        d: Dimension of the context vector that the algorithm should expect.
        alpha: Parameter of the LinUCB algorithm that controls exploration.
        A: Linear regression coefficient matrices.
        b: Vectors of independent terms of the linear regressions.
    """
    def __init__(self, n_arms: int, context_dim: int, alpha: float = 0.1) -> None:
        """Initialisation of a class instance.

        Args:
            n_arms: Number of arms of the bandit problem.
            context_dim: Dimension of the context vector that the algorithm should expect.
            alpha: Parameter of the LinUCB algorithm that controls exploration.
        """
        self.n_arms = n_arms
        self.d = context_dim
        self.alpha = alpha

        self.A = [np.identity(self.d) for _ in range(n_arms)] # A_a = d x d
        self.b = [np.zeros((self.d, 1)) for _ in range(n_arms)] # b_a = d x 1

    def select_action(self, x_contexts: np.ndarray) -> np.int64:
        """Method to choose an action or arm following the algorithm policy.

        This method suggests which arm to choose following the equations of the LinUCB algorithm.

        Args:
            x_contexts: Vector with the context for an specific time step. It has shape (d, 1).

        Returns:
            The number of the arm that should be chosen according to the algorithm.
        """
        p_values = []

        for a in range(self.n_arms):
            A_inv = np.linalg.inv(self.A[a])
            theta = A_inv @ self.b[a]
            x = x_contexts[a]
            p = ((x.T @ theta) + self.alpha * np.sqrt((x.T @ A_inv @ x)))[0, 0]
            p_values.append(p)

        return np.argmax(p_values)

    def update(self, action: int, x: np.ndarray, reward: float) -> None:
        """Update method of the model.

        This method is responsible for updating the parameters of the linear regression of the model after each time
        step (after an action is selected and the reward of that choice is returned).

        Args:
            action: Action or arm chosen.
            x: Context associated to that time step.
            reward: Reward returned from the action chosen.
        """
        self.A[action] += x @ x.T
        self.b[action] += reward * x

```

Figure A.8: Python class implementing the Linear Upper Confidence Bound algorithm.

```

class LinTS:
    """Class containing an implementation of the Linear Thompson Sampling (LinTS) algorithm for Contextual
    Multi-Armed Bandits.

    This implementation initialises the linear regression coefficient matrices to the identity matrix and the vectors of
    independent terms to 0 and considers a multivariate normal distribution  $N(\mu, \sigma)$  as the prior distribution of
    the rewards. Both the coefficient matrices and vectors of independent terms get updated after each time step
    based on the reward obtained.

    Attributes:
        n_arms: Number of arms of the bandit problem.
        d: Dimension of the context vector that the algorithm should expect.
        v: Parameter of the LinTS algorithm that scales the values of the covariance matrix of the multivariate normal
        distribution. It controls exploration.
        A: Linear regression coefficient matrices.
        b: Vectors of independent terms of the linear regressions.
    """
    def __init__(self, n_arms: int, context_dim: int, v: float = 1.0) -> None:
        """Initialisation of a class instance.

        Args:
            n_arms: Number of arms of the bandit problem.
            context_dim: Dimension of the context vector that the algorithm should expect.
            v: Parameter of the LinTS algorithm that scales the values of the covariance matrix of the multivariate
            normal distribution. It controls exploration.
        """
        self.n_arms = n_arms
        self.d = context_dim
        self.v = v

        self.A = [np.identity(self.d) for _ in range(n_arms)] # A_a = d x d
        self.b = [np.zeros((self.d, 1)) for _ in range(n_arms)] # b_a = d x 1

    def select_action(self, x_contexts: np.ndarray) -> np.int64:
        """Method to choose an action or arm following the algorithm policy.

        This method suggests which arm to choose following the equations of the LinTS algorithm.

        Args:
            x_contexts: Vector with the context for an specific time step. It has shape (d, 1).

        Returns:
            The number of the arm that should be chosen according to the algorithm.
        """
        p_values = []

        for a in range(self.n_arms):
            A_inv = np.linalg.inv(self.A[a])
            mu = A_inv @ self.b[a]
            cov = self.v ** 2 * A_inv

            theta_sample = np.random.multivariate_normal(mu.flatten(), cov).reshape(-1, 1)
            x = x_contexts[a]

            p = (theta_sample.T @ x)[0, 0]
            p_values.append(p)

        return np.argmax(p_values)

    def update(self, action: int, x: np.ndarray, reward: float) -> None:
        """Update method of the model.

        This method is responsible for updating the parameters of the linear regression of the model after each time
        step (after an action is selected and the reward of that choice is returned).

        Args:
            action: Action or arm chosen.
            x: Context associated to that time step.
            reward: Reward returned from the action chosen.
        """
        self.A[action] += x @ x.T
        self.b[action] += reward * x

```

Figure A.9: Python class implementing the Linear Thompson Sampling algorithm.

```

rewards_runs = []
regrets_runs = []
accuracies = []
for i in range(n_runs):
    policies = {
        "random": EpsilonGreedy(n_arms=n_arms, epsilon=1.),
        "epsilon_greedy": EpsilonGreedy(n_arms=n_arms, epsilon=0.1),
        "ucb": UCB(n_arms=n_arms, alpha=0.25),
        "thompson_sampling": ThompsonSampling(n_arms=n_arms),
        "linucb": LinUCB(n_arms=n_arms, context_dim=10, alpha=0.75),
        "lints": LinTS(n_arms=n_arms, context_dim=10, v=0.3),
    }
    cum_reward = [0] * len(policies)
    cum_regret = [0] * len(policies)
    accuracy = [0] * len(policies)
    rewards = [[0] for _ in range(len(policies))]
    regrets = [[0] for _ in range(len(policies))]
    for i in range(n_interactions):
        user = np.random.choice(users)
        start = pd.Timestamp('2025-01-01 00:00:00')
        end = pd.Timestamp('2025-12-31 23:59:59')
        timestamp = start + pd.to_timedelta(np.random.uniform(0, 1) * (end - start))

        # Create context vectors
        context_df = products_df.assign(**user)
        context_df = context_df.loc[:, ['age', 'gender', 'preferred_category', 'category']]
        # List of categorical columns
        categorical_cols = ['gender', 'preferred_category', 'category']
        categories = [['M', 'F'], product_categories, product_categories]
        # Create the column transformer
        column_transformer = ColumnTransformer(
            transformers=[
                ('onehot', OneHotEncoder(sparse_output=False, categories=categories, drop='first'), categorical_cols)
            ],
            remainder='passthrough' # or 'passthrough' to keep non-categorical columns
        )
        # Fit and transform the data
        encoded_array = column_transformer.fit_transform(context_df)
        x_contexts = list(encoded_array[:, :, None])

        cat_splits = (['technology', 'sports'], ['fashion', 'books', 'home'])
        interactions = simulate_interactions(user, products_df, timestamp, cat_splits)

        for j, key in enumerate(policies.keys()):
            recommendation = policies[key].select_action(x_contexts)
            product = interactions.iloc[recommendation, 0]
            click = interactions.iloc[recommendation, -1]

            if recommendation == int(interactions.loc[interactions['ctr'].idxmax()]['product_id']):
                accuracy[j] += 1
                cum_reward[j] += click
                cum_regret[j] += interactions.iloc[:, -1].max() - click
                rewards[j].append(cum_reward[j])
                regrets[j].append(cum_regret[j])

            policies[key].update(recommendation, x_contexts[recommendation], click)
    accuracies.append(accuracy)
    rewards_runs.append(rewards)
    regrets_runs.append(regrets)

```

Figure A.10: Simultaneous training of all algorithms.